LAB 03

EXERCISE:

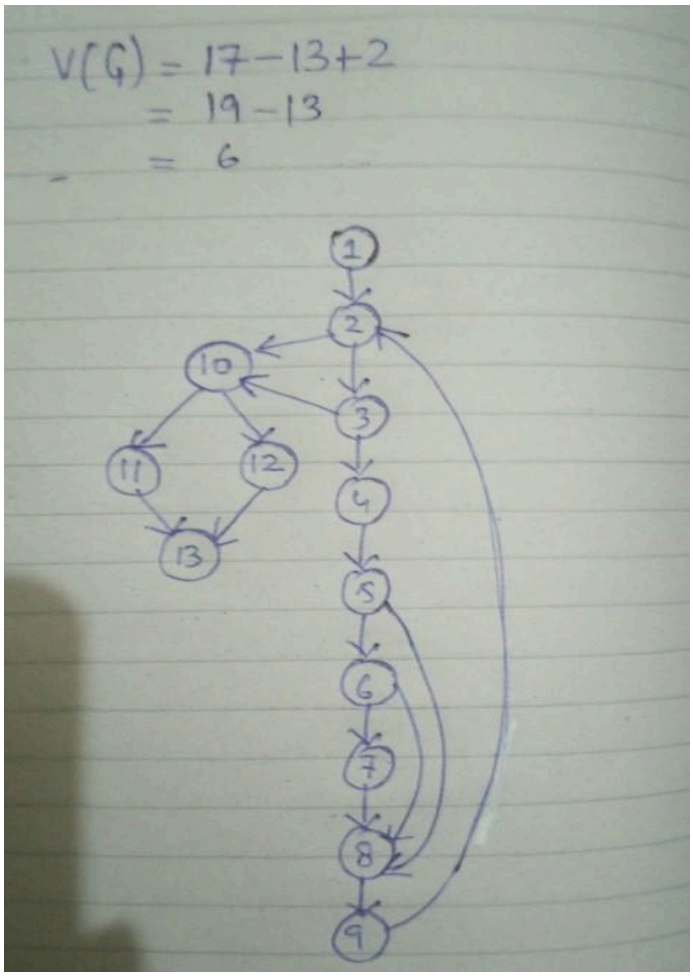
Q1. Consider the code given below. Calculate cyclomatic complexity and no. of linearly independent paths and Design Testcases for each path.

```
#include <iostream>
using namespace std;
int main() {
        int totalInput = 0, totalValid = 0, sum = 0;
        int value[100];
        int minimum = 10, maximum = 50;
        cout << "Enter values (-999 to stop): " << endl;
        while (true) {
        cin >> value[totalInput];
        if (value[totalInput] == -999 || totalInput >= 100) {
        break; }
        totalInput++; }
        for (int i = 0; i < totalInput; i++) {
        if (value[i] >= minimum && value[i] <= maximum) {
        totalValid++;
        sum += value[i]; } }
        double average;
        if (totalValid > 0) {
    average = static_cast<double>(sum) / totalValid; } else {
        average = -999; }
        cout << "Average: " << average << endl;
        return 0; }
```

Test Summary Report

Project Name	Path Coverage Analysis
Test Suite ID	Path_Coverage_001
Test Title	Verifying path execution for differnt scenarios
Test Priority	Med
Module Name	Input_Validation
Designed By	Javeria Shahid
Designed Date	1/2/2025
Executed By	Team
Executed Date	1/2/2025
Description of Test	Validate different input scenarios.

CONTROL FLOW GRAPH



S.No	Test Case	Test Steps	Test Data	Expected Output	Actual Output	Status
TC-001	Verify valid input handling	1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 2	Value = 30,20	Valid input processed	Valid input processed	Pass
TC-002	Verify maximum boundary condition	1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 8 -> 9 -> 2	Value = **max**,20,30	Valid input processed	Valid input processed	Pass
TC-003	Verify minimum boundary condition	1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 9 -> 2	Value = **min**,20,30	Valid input processed	Valid input processed	Pass
TC-004	Verify invalid inputs handling	1 -> 2 -> 10 -> 11 -> 13	Value = 9,20,60,80	Error message displayed	Error message displayed	Pass
TC-005	Verify loop execution when no input is given	1 -> 2 -> 10 -> 12 -> 13	Value = -999	Loop exits without execution	Loop exits without execution	Pass
TC-006	Verify when loop exceed its limit	1 -> 2 -> 3 -> 10 -> 12 -> 13	Value = 10,20,30,...	Error message displayed	Error message displayed	Pass

TC-001

```
Enter values (-999 to stop):
30
20
-999
Average: 25


=== Code Execution Successful ===
```

TC-002

```
Enter values (-999 to stop):
60
20
30
-999
Average: 25


=== Code Execution Successful ===
```

TC-003

```
Enter values (-999 to stop):
9
20
30
-999
Average: 25


=== Code Execution Successful ===
```

TC-004

```
Enter values (-999 to stop):
9
60
20
80
-999
Average: 20


=== Code Execution Successful ===
```

TC-005

```
Enter values (-999 to stop):
-999
Average: -999


=== Code Execution Successful ===
```

TC-006

```
Enter values (-999 to stop):
13
12
12
12
12
12 ...........
```

Q2: Consider the flow chart given below and design test cases by using statement coverage method, and Independent path method also execute test case using template.

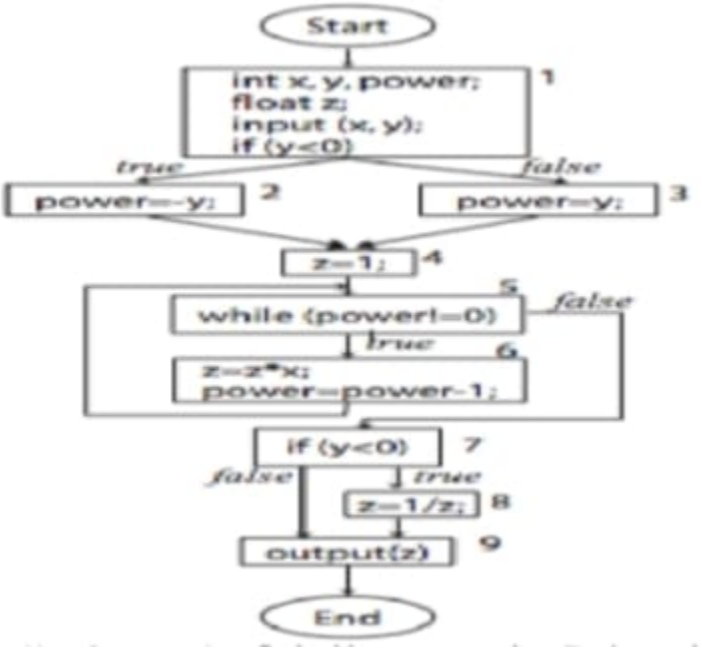
Test Summary Report

Project Name	Power Function Implementation
Test Suite ID	POWER_FUNC_TEST_SUITE_V1
Test Title	Power Function Test Suite
Test Priority	High (Core functionality)
Module Name	Power Function (power(x, y))
Designed by	AbdulRahman Shahvez
Designed Date	2/2/2025
Executed by	Team
Executed Date	2/2/2025
Description of Tests	Tests the power(x, y) function for various exponent and base values, including edge cases. Covers statement and independent path criteria.

Code:

```
function power(x, y) {

let power = 0;

let z = 1; // Initialize z to 1 for all cases

if (y < 0) {

power = -y;

while (power !== 0) {

z = z * x;

power = power - 1;

}

z = 1 / z;

} else {

power = y;

while (power !== 0) {

z = z * x;

power = power - 1;

}

}

return z;

}
```

Control Flow Diagram:



S.No	Test Case	Test Steps	Test Data (x, y)	Expected Output (z)	Actual Output (z)	Pass/Fail
1	Positive Exponent	1. Input x, y , 2. y < 0 is false , 3. power = y , 4. Loop (power > 0) , 5. Loop end (power = 0) , 6. Return z	(2, 3)	8	8	Pass
2	Zero Exponent	1. Input x, y , 2. y < 0 is false , 3. power = y , 4. Loop (power = 0 initially) , 5. Return z	(5, 0)	1	1	Pass
3	Negative Exponent	1. Input x, y , 2. y < 0 is true , 3. power = -y , 4. Loop (power > 0) , 5. Loop end (power = 0) , 6. z = 1/z , 7. Return z	(3, -2)	0.111111111	0.111111	Pass
4	Zero Base	1. Input x, y , 2. y < 0 is false , 3. power = y , 4. Loop (power > 0) , 5. Loop end (power = 0) , 6. Return z	(0, 5)	0	0	Pass
5	Negative Base, Positive Exponent	1. Input x, y , 2. y < 0 is false , 3. power = y , 4. Loop (power > 0) , 5. Loop end (power = 0) , 6. Return z	(-2, 3)	-8	-8	Pass
6	Negative Base, Negative Exponent	1. Input x, y , 2. y < 0 is true , 3. power = -y , 4. Loop (power > 0) , 5. Loop end (power = 0) , 6. z = 1/z , 7. Return z	(-2, -2)	0.25	0.25	Pass
7	Base One	1. Input x, y , 2. y < 0 is false , 3. power = y , 4. Loop (power > 0) , 5. Loop end (power = 0) , 6. Return z	(1, 100)	1	1	Pass
8	Base Ten, Negative Exponent	1. Input x, y , 2. y < 0 is true , 3. power = -y , 4. Loop (power > 0) , 5. Loop end (power = 0) , 6. z = 1/z , 7. Return z	(10, -1)	0.1	0.1	Pass

Test Cases

TC-001

```
Test: 2^3
Expected: 8
Actual: 8
Result: Pass
```

TC-002

```
Test: 5^0
Expected: 1
Actual: 1
Result: Pass
```

TC-003

```
Test: 3^-2
Expected: 0.1111111111111111
Actual: 0.1111111111111111
Result: Pass
```

TC-004

```
Test: 0^5
Expected: 0
Actual: 0
Result: Pass
```

TC-005

```
Test: -2^3
Expected: -8
Actual: -8
Result: Pass
```

TC-006

```
Test: -2^-2
Expected: 0.25
Actual: 0.25
Result: Pass
```

TC-007

```
Test: 1^100
Expected: 1
Actual: 1
Result: Pass
```

TC-008

```
Test: 10^-1
Expected: 0.1
Actual: 0.1
Result: Pass
```

All Paths:

1. 1-2-4-5-6-4-7-8-9
2. 1-2-4-5-6-4-7-9
3. 1-3-4-5-6-4-7-9

S.No	Test Case	Test Steps	Test Data (x, y)	Expected Output (z)	Actual Output (z)	Pass/Fail
TC_001	Path 1: Positive Exponent	1. Input x, y , 2. y < 0 is false , 3. power = y , 4. Loop (power > 0) , 5. Loop end (power = 0) , 6. Return z	(2, 3)	8	8	Pass
TC_002	Path 2: Zero Exponent	1. Input x, y , 2. y < 0 is false , 3. power = y , 4. Loop (power = 0 initially) , 5. Return z	(5, 0)	1	1	Pass
TC_003	Path 3: Negative Exponent	1. Input x, y , 2. y < 0 is true , 3. power = -y , 4. Loop (power > 0) , 5. Loop end (power = 0) , 6. z = 1/z , 7. Return z	(3, -2)	0.111111111	0.111111111	Pass

Test Cases:

TC_001

```
Start: powerInstrumented( 2 , 3 )
Path 2: y < 0 (False)
Loop (positive or zero y): power = 3
Loop (positive or zero y): power = 2
Loop (positive or zero y): power = 1
End: Returning z = 8
Test: 2^3
Expected: 8
Actual: 8
Result: Pass
```

TC_002

```
Start: powerInstrumented( 3 , -2 )
Path 1: y < 0 (True)
Loop (negative y): power = 2
Loop (negative y): power = 1
After division: z = 0.1111111111111111
End: Returning z = 0.1111111111111111
Test: 3^-2
Expected: 0.1111111111111111
Actual: 0.1111111111111111
Result: Pass
```

TC_003

```
Start: powerInstrumented( 5 , 0 )
Path 2: y < 0 (False)
End: Returning z = 1
Test: 5^0
Expected: 1
Actual: 1
Result: Pass
```

Q3: Write a program (Any Language) to find odd or even number. Design and execute test cases using branch coverage method using template /*attach printout here*/

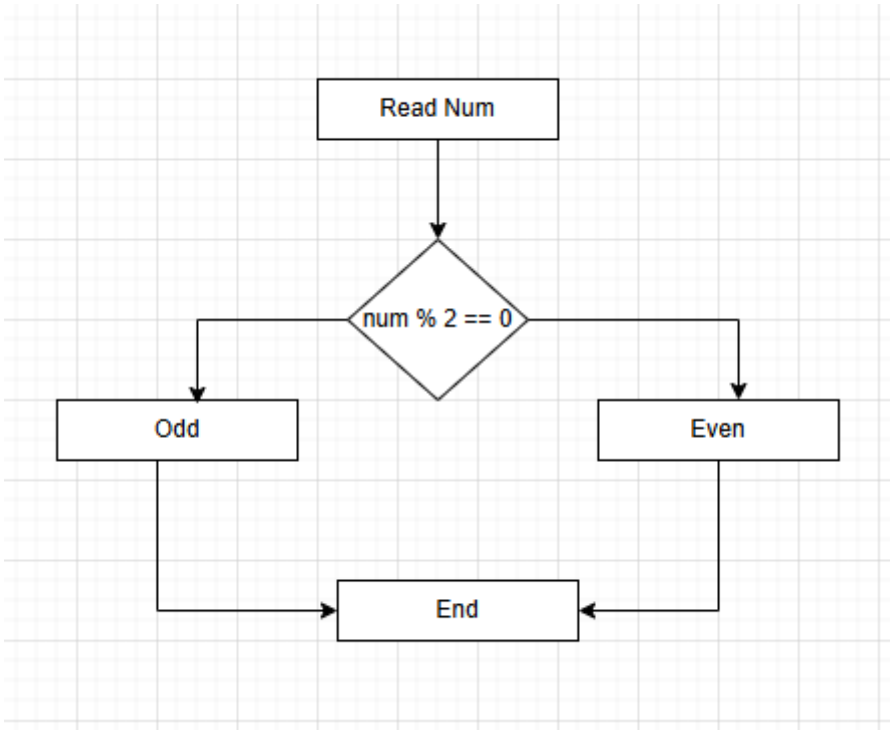
Test Summary Report

Project Name	Odd/Even Number Checker
Test Suite ID	TC-001
Test Title	Verify Odd/Even Number Function
Test Priority	Medium
Module Name	Number Classification
Designed by	Shanila Narejo
Designed Date	22 January
Executed by	Team
Executed Date	22 January
Description of Test	Validate the function to check if a number is odd or even using branch coverage.
PREREQUISITES	
Pre-Condition	User provides a valid integer input
Dependencies	None

Code:

```
#include <iostream>
using namespace std;
int main() {
   int num;
   cout << "Enter a number: ";
   cin >> num;
   if (num % 2 == 0) {
      cout << num << " is an even number." << endl;
   } else {
      cout << num << " is an odd number." << endl;
   }
   return 0;
}
```

Control Flow Diagram



Branch Coverage method:

Branch coverage = (branch covered by TC/ total number of branches) * 100

Sno	Test Case	Test Steps	Test Data	Expected Output	Actual Output	Pass / Fail	Comments
TC-001	Verify branch coverage by using even number as input	compile, Enter num	num = 2	print "2 is even"	print "2 is even"	Pass	50%
TC-002	Verify branch coverage by using odd number as input	compile, Enter num	num = 1	print "1 is odd"	print "1 is odd"	Pass	50%

Test Cases

TC-001

```
Enter a number: 2
2 is an even number.
```

TC-002

```
Enter a number: 1
1 is an odd number.
```

Branch coverage = ½ * 100 = 50%

Q4: Write a program (Any Language) to perform addition, subtraction, multiplication and division. Observe the output and Test it using any one of white box technique .Attached all printout here.

Test Summary Report

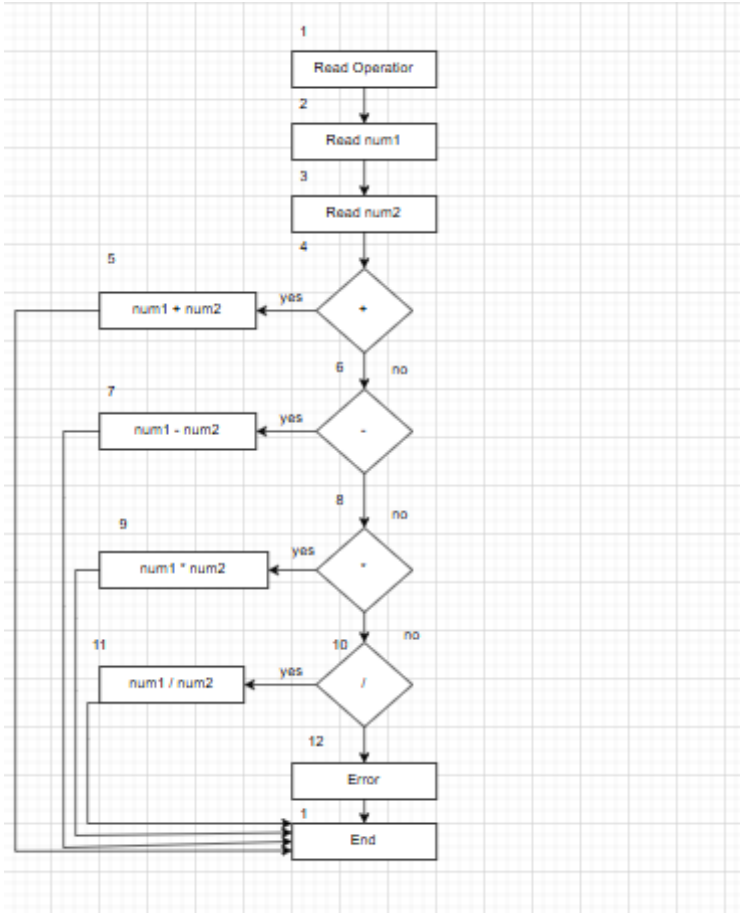
Project Name	Arithmetic Calculator
Test Suite ID	TC-001
Test Title	Validate Basic Arithmetic Operations
Test Priority	High
Module Name	Calculator Functions
Designed by	Shanila Narejo
Designed Date	22 January
Executed by	Team
Executed Date	22 January
Description of Test	Verify the calculator function for all operations using statement coverage.
PREREQUISITES	
Pre-Condition	User provides two valid numeric inputs
Dependencies	None

Code:

```
#include <iostream>
using namespace std;
int main() {
char op;
float num1, num2;
```

```
cout << "Enter operator: +, -, *, /: ";
cin >> op;
cout << "Enter two operands: ";
cin >> num1 >> num2;
switch(op) {
case '+':
cout << num1 << " + " << num2 << " = " << num1 + num2;
break;
case '-':
cout << num1 << " - " << num2 << " = " << num1 - num2;
break;
case '*':
cout << num1 << " * " << num2 << " = " << num1 * num2;
break;
case '/':
cout << num1 << " / " << num2 << " = " << num1 / num2;
break;
default:
// If the operator is other than +, -, * or /, error message is shown
cout << "Error! operator is not correct";
break;
}
return 0;
}
```

Control Flow Diagram



All Paths:

1. 1,2,3,4,5,13
2. 1,2,3,4,6,7,13
3. 1,2,3,4,6,8,9,13
4. 1,2,3,4,6,8,10,11,13
5. 1,2,3,4,6,8,10,12,13

Sno	Test Case	Test Steps	Test Data	Expected Output	Actual Output	Pass / Fail	Comments
TC-001	Verify addition	Enter operator, Enter num1, Enter num2	opertaor = + , num1=2, num2=5	2+5 = 7	2+5 = 7	Pass	
TC-002	Verify subtraction	Enter operator, Enter num1, Enter num2	opertaor = - , num1=3, num2=2	3-2 = 1	3-2 = 1	Pass	
TC-003	Verify multiplication	Enter operator, Enter num1, Enter num2	opertaor = * , num1=5, num2=5	5*5 = 25	5*5 = 25	Pass	
TC-004	Verify division	Enter operator, Enter num1, Enter num2	opertaor = / , num1=2, num2=2	2/2 = 1	2/2 = 1	Pass	
TC-005	Verify error condition	Enter operator, Enter num1, Enter num2	opertaor = & , num1=2, num2=5	Error! The operator is not correct	Error! The operator is not correct	Pass	

Test Cases

TC-001

```
Enter operator: +, -, *, /: +
Enter two operands: 2 5
2 + 5 = 7
```

TC-002

```
Enter operator: +, -, *, /: -
Enter two operands: 3 2
3 - 2 = 1
```

TC-003

```
Enter operator: +, -, *, /: *
Enter two operands: 5 5
5 * 5 = 25
```

TC-004

```
Enter operator: +, -, *, /: /
Enter two operands: 2 2
2 / 2 = 1
```

TC-005

```
Enter operator: +, -, *, /: &
Enter two operands: 2 5
Error! operator is not correct
```